Chapter 5 Review Questions

1. Per-router control means that a routing algorithm runs in each and every router; both forwarding and routing function are constrained within each router. Each router has a routing component that communicates with the routing components in other routers to compute the values for its forwarding table. In such cases, we say that the network control and data planes are implemented monolithically because each router works as an independent entity that implements its own control and data planes.

2. Logically centralized control means that a logically central routing controller computes and distributes the forwarding tables to be used by each and every router, and each router does not compute its forwarding table, unlike the per-router control. In the case of logically centralized control, the data plane and control plane are implemented in separate devices; the control plane is implemented in a central server or multiple servers, and the data plane is implemented in each router.

3. A centralized routing algorithm computes the least-cost path between a source and destination by using complete, global knowledge about the network. The algorithm needs to have the complete knowledge of the connectivity between all nodes and all links' costs. The actual calculation can be run at one site or could be replicated in the routing component of each and every router. A distributed routing algorithm calculates the lease-cost path in an iterative, distributed manner by the routers. With a decentralized algorithm, no node has the complete information about the costs of all network links. Each node begins with only the knowledge of the costs of its own directly attached links, and then through an iterative process of calculation and information exchange with its neighboring nodes, a node gradually calculates the least-cost path to a destination or a set of destinations.

   OSPF protocol is an example of centralized routing algorithm, and BGP is an example of a distributed routing algorithm.

4. In static routing algorithms, routes change very slowly over time, often as a result of human intervention (for example, a human manually editing a router's forwarding table). Dynamic routing algorithms change the routing paths as the network traffic loads or topology change. A dynamic algorithm can be run either periodically or in direct response to topology or link cost changes. While dynamic algorithms are more responsive to network changes, they are also more susceptible to problems such as routing loops and oscillation in routes.

5. The count-to-infinity problem refers to a problem of distance vector routing. The problem means that it takes a long time for a distance vector routing algorithm to converge when there is a link cost increase. For example, consider a network of three nodes x, y, and z. Suppose initially the link costs are c(x,y)=4, c(x,z)=50, and c(y,z)=1. The result of distance-vector routing algorithm says that z's path to x is z→y→ x and the cost is 5(=4+1). When the cost of link (x,y) increases from 4 to 60, it will take 44 iterations of running the distance-vector routing algorithm for node z to

realize that its new least-cost path to x is via its direct link to x, and hence y will also realize its least-cost path to x is via z.

6. Instead, each node begins with only the knowledge of the costs of its own directly attached links. Then, through an iterative process of calculation and exchange of information with its neighboring nodes (that is, nodes that are at the other end of links to which it itself is attached), a node gradually calculates the least-cost path to a destination or set of destinations.

7. Policy: Among ASs, policy issues dominate. It may well be important that traffic originating in a given AS not be able to pass through another specific AS. Similarly, a given AS may want to control what transit traffic it carries between other ASs. Within an AS, everything is nominally under the same administrative control and thus policy issues a much less important role in choosing routes with in AS.

   Scale: The ability of a routing algorithm and its data structures to scale to handle routing to/among large numbers of networks is a critical issue in inter-AS routing. Within an AS, scalability is less of a concern. For one thing, if a single administrative domain becomes too large, it is always possible to divide it into two ASs and perform inter-AS routing between the two new ASs.

   Performance: Because inter-AS routing is so policy oriented, the quality (for example, performance) of the routes used is often of secondary concern (that is, a longer or more costly route that satisfies certain policy criteria may well be taken over a route that is shorter but does not meet that criteria). Indeed, we saw that among ASs, there is not even the notion of cost (other than AS hop count) associated with routes. Within a single AS, however, such policy concerns are of less importance, allowing routing to focus more on the level of performance realized on a route.

8. False.
   With OSPF, a router broadcasts its link-state information to all other routers in the autonomous system to which it belongs, not just to its neighboring routers. This is because with OSPF, each router needs to construct a complete topological map of the entire AS and then locally runs Dijkstra's shortest-path algorithm to determine its least-cost paths to all other nodes in the same AS.

9. An area in an OSPF autonomous system is refers to a set of routers, in which each router broadcasts its link state to all other routers in the same set. An OSPF AS can be configured hierarchically into multiple areas, with each area running its own OSPF link-state routing algorithm. Within each area, one or more area border routers are responsible for routing packets outside the area. The concept of area is introduced for scalability reason, i.e., we would like to build a hierarchical routing for a large scale OSPF AS, and an area is an important building block in hierarchical routing.

10. A subnet is a portion of a larger network; a subnet does not contain a router; its boundaries are defined by the router and host interfaces. A prefix is the network

portion of a CDIRized address; it is written in the form a.b.c.d/x ; A prefix covers one or more subnets. When a router advertises a prefix across a BGP session, it includes with the prefix a number of BGP attributes. In BGP jargon, a prefix along with its attributes is a BGP route (or simply a route).

11. Routers use the AS-PATH attribute to detect and prevent looping advertisements; they also use it in choosing among multiple paths to the same prefix. The NEXT-HOP attribute indicates the IP address of the first router along an advertised path (outside of the AS receiving the advertisement) to a given prefix. When configuring its forwarding table, a router uses the NEXT-HOP attribute.

12. A tier-1 ISP B may not to carry transit traffic between two other tier-1 ISPs, say A and C, with which B has peering agreements. To implement this policy, ISP B would not advertise to A routes that pass through C; and would not advertise to C routes that pass through A.

13. False.
    A BGP router can choose not to add its own identity to the received path and then send that new path on to all of its neighbors, as BGP is a policy-based routing protocol. This can happen in the following scenario. The destination of the received path is some other AS, instead of the BGP router's AS, and the BGP router does not want to work as a transit router.

14. The communication layer is responsible for the communication between the SDN controller and those controlled network devices, via a protocol such as OpenFlow. Through this layer, an SDN controller controls the operation of a remote SDN-enabled switch, host, or other devices, and a device communicates locally-observed events (e.g., a message indicating a link failure) to the controller.

    The network-wide state-management layer provides up-to-date information about state a network's hosts, links, switches, and other SDN-controlled devices. A controller also maintains a copy of the flow tables of the various controlled devices.

    The network-control application layer represents the brain of SDN control plane. The applications at this layer use the APIs provided by a SDN controller to specify and control the data plane in the network devices. For example, a routing network-control application might determine the end-end paths between sources and destinations. Another network application might perform access control.

15. I would implement a new routing protocol at the SDN's network-control application layer, as this is the layer where a routing protocol determines the end-to-end paths between sources and destinations.

16. The following is a list of types of messages flow across a SDN controller's southbound from the controller to the controlled devices. The recipient of these messages is a controlled packet switch.

- Configuration. This message allows the controller to query and set a switch's configuration parameters.
- Modify-state. This message is used by a controller to add/delete or modify entries in the switch's flow table, and to set switch port properties.
- Read-state. This message is used by a controller to collect statistics and counter values from the switch's flow table and ports.
- Send-packet. This message is used by the controller to send a specific packet out of a specified port at the controlled switch.

There are also messages that network-control applications (as senders) send to the controller across the northbound interfaces, for example, messages to read/write network state and flow tables within the state-management layer of the controller.

17. Two types of messages from a controlled device to a controller:
    - Flow-removed message. Its purpose is to inform the controller that a flow table entry has been removed, for example, by a timeout or as the result of a received modify-state message.
    - Port-status message. Its purpose is to inform the controller of a change in port status.

    Two types of messages from a controller to a controlled device:
    - Modify-state. The purpose is to add/delete or modify entries in the switch's flow table, and to set switch port properties.
    - Read-state. The purpose is to collect statistics and counter values rom the switch's flow table and ports.

18. The service abstraction layer allows internal network service applications to communicate with each other. It allows controller components and applications to invoke each other's services and to subscribe to events they generate. This layer also provides a uniform abstract interface to the specific underlying communications protocols in the communication layer, including OpenFlow and SNMP.

19. Echo reply (to ping), type 0, code 0
    Destination network unreachable, type 3 code 0
    Destination host unreachable, type 3, code 1.
    Source quench (congestion control), type 4 code 0.

20.
ICMP warning message (type 11 code 0) and a destination port unreachable ICMP message (type 3 code 3).

21.
A *managing server* is an application, typically with a human in the loop, running in a centralized network management station in a network operation center. It controls the collection, processing, analysis, and/or display of network management information. Actions are initiated in a managing server to control network behavior and a network administrator uses a managing server to interact with the network's devices.

A *managed device* is a piece of network equipment (including its software) that resides on a managed network. A managed device might be a host, router, switch, middlebox, modem, thermometer, or other network-connected device.

A *network management agent* is a process running in a managed device that communicates with a managing server, taking local actions at the managed device under the command and control of the managing server.

Management Information Base (MIB) collects the information associated with those managed objects in a managed network. A MIB object might be a counter, such as the number of IP datagrams discarded at a router due to errors in an IP datagram header, or the number of UDP segments received at a host, or the status information such as whether a particular device is functioning correctly.

22.
GetRequest is a message sent from a managing server to an agent to request the value of one or more MIB objects at the agent's managed device.

SetRequest is a message used by a managing server to set the value of one or more MIB objects in a managed device.

23.
A SNMP trap message is generated as a response to an event happened on a managed device for which the device's managing server requires notification. It is used for notifying a managing server of an exceptional situation (e.g., a link interface going up or down) that has resulted in changes to MIB object values.

Chapter 5 Problems

Problem 1

y-x-u, y-x-v-u, y-x-w-u, y-x-w-v-u,
y-w-u, y-w-v-u, y-w-x-u, y-w-x-v-u, y-w-v-x-u,
y-z-w-u, y-z-w-v-u, y-z-w-x-u, y-z-w-x-v-u, y-z-w-v-x-u,

Problem 2

x to z:
x-y-z, x-y-w-z,
x-w-z, x-w-y-z,
x-v-w-z, x-v-w-y-z,
x-u-w-z, x-u-w-y-z,
x-u-v-w-z, x-u-v-w-y-z

z to u:
z-w-u,
z-w-v-u, z-w-x-u, z-w-v-x-u, z-w-x-v-u, z-w-y-x-u, z-w-y-x-v-u,
z-y-x-u, z-y-x-v-u, z-y-x-w-u, z-y-x-w-y-u, z-y-x-v-w-u,
z-y-w-v-u, z-y-w-x-u, z-y-w-v-x-u, z-y-w-x-v-u, z-y-w-y-x-u, z-y-w-y-x-v-u

z to w:
z-w, z-y-w, z-y-x-w, z-y-x-v-w, z-y-x-u-w, z-y-x-u-v-w, z-y-x-v-u-w

Problem 3

Step	*N'*	*D(t),p(t)*	*D(u),p(u)*	*D(v),p(v)*	*D(w),p(w)*	*D(y),p(y)*	*D(z),p(z)*
0	x	∞	∞	3,x	6,x	6,x	8,x
1	xv	7,v	6,v	3,x	6,x	6,x	8,x
2	xvu	7,v	6,v	3,x	6,x	6,x	8,x
3	xvuw	7,v	6,v	3,x	6,x	6,x	8,x
4	xvuwy	7,v	6,v	3,x	6,x	6,x	8,x
5	xvuwyt	7,v	6,v	3,x	6,x	6,x	8,x
6	xvuwytz	7,v	6,v	3,x	6,x	6,x	8,x

Problem 4

a)

Step	*N'*	*D(x), p(x)*	*D(u),p(u)*	*D(v),p(v)*	*D(w),p(w)*	*D(y),p(y)*	*D(z),p(z)*
0	t	∞	2,t	4,t	∞	7,t	∞
1	tu	∞	2,t	4,t	5,u	7,t	∞
2	tuv	7,v	2,t	4,t	5,u	7,t	∞
3	tuvw	7,v	2,t	4,t	5,u	7,t	∞
4	tuvwx	7,v	2,t	4,t	5,u	7,t	15,x
5	tuvwxy	7,v	2,t	4,t	5,u	7,t	15,x
6	tuvwxyz	7,v	2,t	4,t	5,u	7,t	15,x

b)

Step	*N'*	*D(x), p(x)*	*D(t),p(t)*	*D(v),p(v)*	*D(w),p(w)*	*D(y),p(y)*	*D(z),p(z)*
	u	∞	2,u	3,u	3,u	∞	∞
	ut	∞	2,u	3,u	3,u	9,t	∞
	utv	6,v	2,u	3,u	3,u	9,t	∞
	utvw	6,v	2,u	3,u	3,u	9,t	∞
	utvwx	6,v	2,u	3,u	3,u	9,t	14,x
	utvwxy	6,v	2,u	3,u	3,u	9,t	14,x
	utvwxyz	6,v	2,u	3,u	3,u	9,t	14,x

c)

Step	*N'*	*D(x), p(x)*	*D(u),p(u)*	*D(t),pt)*	*D(w),p(w)*	*D(y),p(y)*	*D(z),p(z)*
	v	3,v	3,v	4,v	4,v	8,v	∞
	vx	3,v	3,v	4,v	4,v	8,v	11,x
	vxu	3,v	3,v	4,v	4,v	8,v	11,x
	vxut	3,v	3,v	4,v	4,v	8,v	11,x
	vxutw	3,v	3,v	4,v	4,v	8,v	11,x
	vxutwy	3,v	3,v	4,v	4,v	8,v	11,x
	vxutwyz	3,v	3,v	4,v	4,v	8,v	11,x

d)

Step	*N'*	*D(x), p(x)*	*D(u),p(u)*	*D(v),p(v)*	*D(t),p(t)*	*D(y),p(y)*	*D(z),p(z)*
	w	6,w	3,w	4,w	∞	∞	∞
	wu	6,w	3,w	4,w	5,u	∞	∞
	wuv	6,w	3,w	4,w	5,u	12,v	∞
	wuvt	6,w	3,w	4,w	5,u	12,v	∞
	wuvtx	6,w	3,w	4,w	5,u	12,v	14,x
	wuvtxy	6,w	3,w	4,w	5,u	12,v	14,x
	wuvtxyz	6,w	3,w	4,w	5,u	12,v	14,x

e)

Step	*N'*	*D(x), p(x)*	*D(u),p(u)*	*D(v),p(v)*	*D(w),p(w)*	*D(t),p(t)*	*D(z),p(z)*
	y	6,y	∞	8,y	∞	7,y	12,y
	yx	6,y	∞	8,y	12,x	7,y	12,y
	yxt	6,y	9,t	8,y	12,x	7,y	12,y
	yxtv	6,y	9,t	8,y	12,x	7,y	12,y
	yxtvu	6,y	**9,t**	8,y	12,x	7,y	12,y
	yxtvuw	6,y	9,t	8,y	12,x	7,y	12,y
	yxtvuwz	6,y	9,t	8,y	12,x	7,y	12,y

f)

Step	*N'*	*D(x), p(x)*	*D(u),p(u)*	*D(v),p(v)*	*D(w),p(w)*	*D(y),p(y)*	*D(t),p(t)*
	z	8,z	∞	∞	∞	12,z	∞
	zx	8,z	∞	11,x	14,x	12,z	∞
	zxv	8,z	14,v	**11,x**	14,x	12,z	15,v
	zxvy	8,z	14,v	11,x	14,x	**12,z**	15,v
	zxvyu	8,z	**14,v**	11,x	14,x	12,z	15,v
	zxvyuw	8,z	14,v	11,x	**14,x**	12,z	15,v
	zxvyuwt	8,z	14,v	11,x	**14,x**	12,z	15,v

Problem 5

		Cost to u	v	x	y	z
	v	∞	∞	∞	∞	∞
From	x	∞	∞	∞	∞	∞
	z	∞	6	2	∞	0

		Cost to u	v	x	y	z
	v	1	0	3	∞	6
From	x	∞	3	0	3	2
	z	7	5	2	5	0

		Cost to u	v	x	y	z
	v	1	0	3	3	5
From	x	4	3	0	3	2
	z	6	5	2	5	0

		Cost to u	v	x	y	z
	v	1	0	3	3	5
From	x	4	3	0	3	2
	z	6	5	2	5	0

Problem 6

The wording of this question was a bit ambiguous. We meant this to mean, "the number of iterations from when the algorithm is run for the first time" (that is, assuming the only information the nodes initially have is the cost to their nearest neighbors). We assume that the algorithm runs synchronously (that is, in one step, all nodes compute their distance tables at the same time and then exchange tables).

At each iteration, a node exchanges distance tables with its neighbors. Thus, if you are node A, and your neighbor is B, all of B's neighbors (which will all be one or two hops from you) will know the shortest cost path of one or two hops to you after one iteration (i.e., after B tells them its cost to you).

Let d be the "diameter" of the network - the length of the longest path without loops between any two nodes in the network. Using the reasoning above, after $d-1$ iterations, all nodes will know the shortest path cost of d or fewer hops to all other nodes. Since any path with greater than d hops will have loops (and thus have a greater cost than that path with the loops removed), the algorithm will converge in at most $d-1$ iterations.

ASIDE: if the DV algorithm is run as a result of a change in link costs, there is no a priori bound on the number of iterations required until convergence unless one also specifies a bound on link costs.

Problem 7

a) Dx(w) = 2, Dx(y) = 4, Dx(u) = 7

b) First consider what happens if c(x,y) changes. If c(x,y) becomes larger or smaller (as long as c(x,y) >=1) , the least cost path from x to u will still have cost at least 7. Thus a change in c(x,y) (if c(x,y)>=1) will not cause x to inform its neighbors of any changes.
If c(x,y)= δ<1, then the least cost path now passes through y and has cost δ+6.

Now consider if c(x,w) changes. If c(x,w) = ε ≤ 1, then the least-cost path to u continues to pass through w and its cost changes to 5 + ε; x will inform its neighbors of this new cost. If c(x,w) = δ > 6, then the least cost path now passes through y and has cost 11; again x will inform its neighbors of this new cost.

c) Any change in link cost c(x,y) (and as long as c(x,y) >=1) will not cause x to inform its neighbors of a new minimum-cost path to u .

Problem 8

Node x table

		Cost to		
		x	y	z
	x	0	3	4
From	y	∞	∞	∞
	z	∞	∞	∞

		Cost to		
		x	y	z
	x	0	3	4
From	y	3	0	6
	z	4	6	0

Node y table

		Cost to		
		x	y	z
	x	∞	∞	∞
From	y	3	0	6
	z	∞	∞	∞

		Cost to		
		x	y	z
	x	0	3	4
From	y	3	0	6
	z	4	6	0

Node z table

		Cost to		
		x	y	z
	x	∞	∞	∞
From	y	∞	∞	∞
	z	4	6	0

		Cost to		
		x	y	z
	x	0	3	4
From	y	3	0	6
	z	4	6	0

Problem 9

No. loops involving three or more nodes will not be detected by the poisoned reverse technique.Problem 33

At each step, each updating of a node's distance vectors is based on the Bellman-Ford equation, i.e., only decreasing those values in its distance vector. There is no increasing in values. If no updating, then no message will be sent out. Thus, D(x) is non-increasing. Since those costs are finite, then eventually distance vectors will be stabilized in finite steps.

Problem 10

At each step, each updating of a node's distance vectors is based on the Bellman-Ford equation, i.e., only decreasing those values in its distance vector. There is no increasing in values. If no updating, then no message will be sent out. Thus, D(x) is non-increasing. Since those costs are finite, then eventually distance vectors will be stabilized in finite steps.

Problem 11

a)

Router z	Informs w, $D_z(x)=\infty$
	Informs y, $D_z(x)=6$
Router w	Informs y, $D_w(x)=\infty$
	Informs z, $D_w(x)=5$
Router y	Informs w, $D_y(x)=4$
	Informs z, $D_y(x)=4$

b) Yes, there will be a count-to-infinity problem. The following table shows the routing converging process. Assume that at time t0, link cost change happens. At time t1, y updates its distance vector and informs neighbors w and z. In the following table, "→" stands for "informs".

time	t0	t1	t2	t3	t4
Z	→ w, $D_z(x)=\infty$ → y, $D_z(x)=6$		No change	→ w, $D_z(x)=\infty$ → y, $D_z(x)=11$	
W	→ y, $D_w(x)=\infty$ → z, $D_w(x)=5$		→ y, $D_w(x)=\infty$ → z, $D_w(x)=10$		No change
Y	→ w, $D_y(x)=4$ → z, $D_y(x)=4$	→ w, $D_y(x)=9$ → z, $D_y(x)=\infty$		No change	→ w, $D_y(x)=14$ → z, $D_y(x)=\infty$

We see that w, y, z form a loop in their computation of the costs to router x. If we continue the iterations shown in the above table, then we will see that, at t27, z detects

that its least cost to x is 50, via its direct link with x. At t29, w learns its least cost to x is 51 via z. At t30, y updates its least cost to x to be 52 (via w). Finally, at time t31, no updating, and the routing is stabilized.

time	t27	t28	t29	t30	t31
Z	→ w, $D_z(x)=50$ → y, $D_z(x)=50$				via w, ∞ via y, 55 via z, 50
W		→ y, $D_w(x)=\infty$ → z, $D_w(x)=50$	→ y, $D_w(x)=51$ → z, $D_w(x)= \infty$		via w, ∞ via y, ∞ via z, 51
Y		→ w, $D_y(x)=53$ → z, $D_y(x)= \infty$		→ w, $D_y(x)= \infty$ → z, $D_y(x)= 52$	via w, 52 via y, 60 via z, 53

c) cut the link between y and z.

Problem 12

O(|N||E|), where N is the set of hosts and E is the set of edges or links.

Problem 13

The chosen path is not necessarily the shortest AS-path. Recall that there are many issues to be considered in the route selection process. It is very likely that a longer loop-free path is preferred over a shorter loop-free path due to economic reason. For example, an AS might prefer to send traffic to one neighbor instead of another neighbor with shorter AS distance.

Problem 14

a) eBGP

b) iBGP

c) eBGP

d) iBGP

Problem 15

a) I1 because this interface begins the least cost path from 1d towards the gateway router 1c.

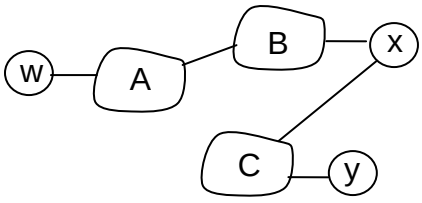
b) I2. Both routes have equal AS-PATH length but I2 begins the path that has the closest NEXT-HOP router.

c) I1. I1 begins the path that has the shortest AS-PATH.

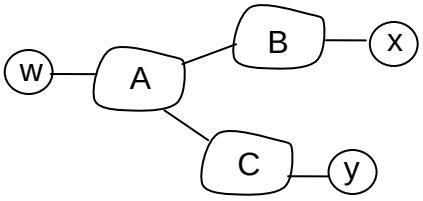
Problem 16

One way for C to force B to hand over all of B's traffic to D on the east coast is for C to only advertise its route to D via its east coast peering point with C.

Problem 17



X's view of the topology



W's view of the topology

In the above solution, X does not know about the AC link since X does not receive an advertised route to w or to y that contain the AC link (i.e., X receives no advertisement containing both AS A and AS C on the path to a destination.

Problem 18

BitTorrent file sharing and Skype P2P applications.
Consider a BitTorrent file sharing network in which peer 1, 2, and 3 are in stub networks W, X, and Y respectively. Due the mechanism of BitTorrent's file sharing, it is quire possible that peer 2 gets data chunks from peer 1 and then forwards those data chunks to 3. This is equivalent to B forwarding data that is finally destined to stub network Y.

Problem 19

A should advise to B two routes, AS-paths A-W and A-V.
A should advise to C only one route, A-V.
C receives AS paths: B-A-W, B-A-V, A-V.

Problem 20

Since Z wants to transit Y's traffic, Z will send route advertisements to Y. In this manner, when Y has a datagram that is destined to an IP that can be reached through Z, Y will have the option of sending the datagram through Z. However, if Z advertizes routes to Y, Y can re-advertize those routes to X. Therefore, in this case, there is nothing Z can do from preventing traffic from X to transit through Z.

Problem 21

Request response mode will generally have more overhead (measured in terms of the number of messages exchanged) for several reasons. First, each piece of information received by the manager requires two messages: the poll and the response. Trapping generates only a single message to the sender. If the manager really only wants to be notified when a condition occurs, polling has more overhead, since many of the polling messages may indicate that the waited-for condition has not yet occurred. Trapping generates a message only when the condition occurs.

Trapping will also immediately notify the manager when an event occurs. With polling, the manager needs will need to wait for half a polling cycle (on average) between when the event occurs and the manager discovers (via its poll message) that the event has occurred.

If a trap message is lost, the managed device will not send another copy. If a poll message, or its response, is lost the manager would know there has been a lost message (since the reply never arrives). Hence the manager could repoll, if needed.

Problem 22

Often, the time when network management is most needed is in times of stress, when the network may be severely congested and packets are being lost. With SNMP running over TCP, TCP's congestion control would cause SNMP to back-off and stop sending messages at precisely the time when the network manager needs to send SNMP messages.